

2026 INTERVIEW GUIDE

30 LangGraph Interview Questions

with Answers

Spoken answers with real code. The 5 areas agent interviews test in 2026.



Naved Khan
Senior Gen-AI Engineer

+ Follow

WHY THIS MATTERS NOW

In 2026, LangGraph stopped being optional.

LangChain's own agents now run on the LangGraph runtime. It became the default way to ship agents, and interviews caught up fast. Checkpointers, interrupts, and state reducers are standard questions now.



Naved Khan
Senior Gen-AI Engineer

+ Follow

5 areas. 30 questions.

1 **Fundamentals**

Q1-6

2 **State & Memory**

Q7-12

3 **Control Flow**

Q13-18

4 **Human-in-the-Loop**

Q19-24

5 **Production**

Q25-30

State and human-in-the-loop are where senior candidates separate from juniors. That's where this guide goes deepest.



Naved Khan

Senior Gen-AI Engineer

+ Follow

Q1 Why does an agent need a graph instead of a chain?

- ▶ A chain runs once, left to right. An agent has to act, observe the result, and decide again. That's a **loop**, and a chain can't express cycles.
- ▶ LangGraph makes the control flow **explicit code**: nodes, edges, state I can read, test, and constrain, instead of hoping a prompt behaves.

Q2 What are the core primitives?

- ▶ **State**: a typed schema every node reads and writes. **Nodes**: plain functions that do the work. **Edges**: fixed or conditional routing between them.
- ▶ I define them on a StateGraph builder, mark **START** and **END**, and compile. The compiled graph is a runnable I can invoke or stream.



Naved Khan
Senior Gen-AI Engineer

+ Follow

Q3 How does state actually move through the graph?

- ▶ Every node receives the current state and returns a **partial update**, not a mutation. LangGraph merges updates at the end of each step.
- ▶ That keeps nodes close to pure functions, which is why graphs are **testable and replayable**. I'd say that part out loud, it's the design insight.

Q4 What is a reducer, and why does `add_messages` exist?

- ▶ A reducer defines **how updates merge** into a state key. Default is overwrite. For chat history, overwriting would destroy the conversation.
- ▶ **`add_messages`** appends new messages and updates existing ones by ID. Custom reducers do the same for any accumulating key.

```
class State(TypedDict):  
    messages: Annotated[list, add_messages]
```



Naved Khan

Senior Gen-AI Engineer

+ Follow

Q5 LangGraph vs CrewAI or AutoGen. How do you choose?

- ▶ They sit at different altitudes. LangGraph is a **low-level orchestration runtime**: explicit state, durability, full control. CrewAI is a role-based abstraction, faster to start, less control.
- ▶ The 2026 data point I'd drop: LangChain's own prebuilt agents **run on the LangGraph runtime** now. It's infrastructure, not a framework choice.

Q6 What does `create_react_agent` actually build?

- ▶ A complete tool-calling loop: model node, tool node, and a conditional edge that **exits when the model stops calling tools**.
- ▶ I'd add that I can rebuild it by hand in about 20 lines, because interviewers often ask exactly that as the follow-up.

```
agent = create_react_agent(model, tools=[search, calc])
```



Naved Khan
Senior Gen-AI Engineer

+ Follow

Q7 Short-term vs long-term memory in LangGraph?

- ▶ Short-term is the **checkpoint**: thread-scoped state, so one conversation can pause and resume with full context.
- ▶ Long-term is the **Store**: namespaced key-value memory that survives across threads, like a user's preferences visible to every future conversation.

Q8 What exactly does a checkpoint save, and when?

- ▶ A **snapshot of the full state after every step**, keyed by thread. That single mechanism is what enables resume, replay, time travel, and human-in-the-loop.
- ▶ MemorySaver for development, **Postgres in production**. Saying that pair signals you've deployed one.

```
graph = builder.compile(checkpointer=PostgresSaver(conn))
```



Naved Khan
Senior Gen-AI Engineer

+ Follow

Q9 What is thread_id and why does it matter?

- ▶ It's the **session key**. Same thread_id continues a conversation from its last checkpoint. New thread_id starts clean.
- ▶ In production it maps to something real: a user session, a support ticket, a workflow run.

```
graph.invoke(inputs, config={"configurable": {"thread_id": "42"}})
```

Q10 The message history keeps growing. What do you do?

- ▶ Three tools: **trim_messages** before each model call, a **summarization node** that compresses old turns, and RemoveMessage to prune state itself.
- ▶ I treat context as a **budget**, the same discipline as RAG. Unbounded history is a cost and quality bug, not a feature.

**Naved Khan**

Senior Gen-AI Engineer

[+ Follow](#)

Q11 What is time travel, practically?

- ▶ Every checkpoint is addressable. I can pull the state history, **fork from any step** with modified state, and re-run from there.
- ▶ Practically it's **debugging**: reproduce the exact state where the agent went wrong, fix one value, and test the counterfactual.

Q12 How would you personalize an agent across sessions?

- ▶ The **Store**: I write memories under a namespace like the user ID, and read them at the start of every new thread.
- ▶ With **semantic search over the store**, the agent recalls relevant facts, not everything. Write selectively: store decisions and preferences, not transcripts.

**Naved Khan**

Senior Gen-AI Engineer

+ Follow

Q13 How do conditional edges work?

- ▶ A **router function** inspects the state and returns the name of the next node. The LLM decides the content, **my code decides the routes**.
- ▶ That separation is the whole point: the set of possible paths is fixed and reviewable, even when the model is unpredictable.

```
builder.add_conditional_edges("agent", route_fn)
```

Q14 What does Command add over normal edges?

- ▶ A node can return **Command(update=..., goto=...)**: change state and choose the next node in one move.
- ▶ It's how dynamic **multi-agent handoffs** work: an agent decides mid-task to pass control, with context attached, without pre-wiring every path.

**Naved Khan**

Senior Gen-AI Engineer

[+ Follow](#)

Q15 How do you run work in parallel inside a graph?

- ▶ For a known fan-out, multiple edges from one node run branches in the same step, and **reducers merge the results**.
- ▶ For a dynamic fan-out, **Send** spawns one node instance per item, each with its own payload. That's map-reduce inside the graph, like grading 20 documents at once.

```
return [Send("grade", {"doc": d}) for d in state["docs"]]
```

Q16 What are subgraphs for?

- ▶ A whole graph becomes **one node** in a parent graph. I use them to encapsulate a skill or a team member with its own internal state.
- ▶ Shared state keys are the interface, private keys stay internal. It's the module system for agents.

**Naved Khan**

Senior Gen-AI Engineer

[+ Follow](#)

Q17 Which multi-agent architecture would you pick?

- ▶ I start with a **supervisor**: one router agent delegating to specialized workers. Predictable, easy to debug.
- ▶ I move to **swarm-style handoffs with Command** when agents genuinely need to pass control sideways. Hierarchies of subgraphs come last, only at real scale.

Q18 What stops an agent from looping forever?

- ▶ The built-in **recursion_limit** kills the run with GraphRecursionError past the step budget.
- ▶ But I don't rely on it alone: I keep **loop counters in state** and hard budgets on tool calls and cost. The limit is a circuit breaker, not a strategy.

**Naved Khan**

Senior Gen-AI Engineer

[+ Follow](#)

Q19 How does interrupt() actually work?

- ▶ Called inside a node, it **pauses the run** and surfaces a payload to the caller. The graph sleeps in its checkpoint, holding no compute.
- ▶ Resume happens with **Command(resume=value)**, and that value becomes interrupt()'s return inside the node. None of it works without a checkpointer.

```
decision = interrupt({"approve": plan})  
# later: graph.invoke(Command(resume="yes"), config)
```

Q20 Where do you put approval gates?

- ▶ Before **writes**, not reads: payments, emails, deletions, deployments. Reading data rarely needs a human.
- ▶ The human can approve, **edit the tool arguments**, or reject with feedback, and the edited version flows back through the resume.



Naved Khan

Senior Gen-AI Engineer

+ Follow

Q21 What does "durable execution" give you?

- ▶ State persists at every step, so a crash, a deploy, or a week-long pause **resumes from the last checkpoint**, not from scratch.
- ▶ The senior nuance: recovery is **not exactly-once for side effects**. A resumed node can re-run, so I add idempotency keys anywhere money or emails move.

Q22 A node keeps failing on a flaky API. Your options?

- ▶ A **retry policy on that node** with backoff handles the transient stuff.
- ▶ For expected failures, I return the error **into state** and route it to a handler node. The failure becomes a visible path in the graph, not a stack trace.



Naved Khan
Senior Gen-AI Engineer

+ Follow

Q23 What streaming modes exist, and why care?

- ▶ **values** streams full state, **updates** streams each node's delta, **messages** streams LLM tokens. They can combine.
- ▶ An agent that thinks for 30 seconds in silence feels broken. Streaming progress is a **UX requirement**, not a nice-to-have.

Q24 How do you test a graph?

- ▶ Nodes are functions, so I **unit test them with a fixed state and a fake model**, asserting on the returned update.
- ▶ Then integration: invoke the compiled graph and assert on the **path it took**, not just the final answer. Evals on real traces close the loop.

**Naved Khan**

Senior Gen-AI Engineer

+ Follow

Q25 How do you deploy LangGraph?

- ▶ Two honest options: **LangGraph Platform** for managed infra with Studio debugging, or **self-host**: a containerized FastAPI service with a Postgres checkpointer.
- ▶ Self-hosting scales horizontally because instances share the **checkpoint store**. Any replica can resume any thread.

Q26 The agent feels slow. Where do you look?

- ▶ First, **parallelize** independent nodes and stream tokens so perceived latency drops immediately.
- ▶ Then **right-size models per node**: routing and extraction on a small model, synthesis on the big one. Most graphs overpay on every hop.

**Naved Khan**

Senior Gen-AI Engineer

+ Follow

Q27 How do you keep agent costs under control?

- ▶ **Budget counters in state:** tokens, tool calls, and loops all have caps the graph enforces.
- ▶ Plus the same levers as any LLM system: trimmed context, cached stable prefixes, and a **model cascade** so cheap queries stay cheap.

Q28 What does observability look like for a graph?

- ▶ **Traces per node and per step**, so I can see exactly which hop burned the tokens or made the bad call.
- ▶ And because state is checkpointed, incident review is **replay**: load the thread, walk the steps, find the turn where it went wrong.



Naved Khan
Senior Gen-AI Engineer

+ Follow

Q29 When would you NOT use LangGraph?

- ▶ A single LLM call or a fixed pipeline is **just code**. No cycles, no persistence, no humans in the loop means the framework is overhead.
- ▶ LangGraph earns its keep the moment I need **state, loops, pauses, or durability**. Before that, simpler wins.

Q30 What actually changed with LangGraph 1.0?

- ▶ A **stable core API**, plus production features: durability modes, node caching, deferred nodes.
- ▶ The bigger shift is positioning: with LangChain agents standardized on this runtime, LangGraph became the **boring, safe choice**. In infrastructure, boring is a compliment.

**Naved Khan**

Senior Gen-AI Engineer

+ Follow

Preparing for AI interviews?



Naved Khan

Senior Gen-AI Engineer

I post practical GenAI interview prep, **RAG, agents, and production AI**, every week.

Follow along if that's useful to you.